

QUESTION NO 1:

COMPARISON OF DDA AND BRESENHAM'S ALGORITHMS

Both the Digital Differential Analyzer (DDA) and Bresenham's Line Algorithm are incremental algorithms used in computer graphics to "rasterize" a line—that is, to determine which pixels on a 2D grid should be illuminated to approximate a straight line between two given points.¹ However, they achieve this in fundamentally different ways.

CRITERIA	INCREMENTAL LINE ALGORITHM (DDA)	BRESENHAM'S LINE ALGORITHM
Working Principle	DDA calculates the slope (m) of the line. Based on the slope, it increments one coordinate by 1 (the "driving axis") and the other coordinate by a floating-point value (m or $\frac{1}{m}$) at each step. The resulting coordinate is then rounded to the nearest integer to select the pixel.	Bresenham's algorithm uses an integer decision parameter (or error term). At each step, it increments the "driving axis" by 1 and checks the sign of this parameter. This check efficiently determines which of the two <i>next possible pixels</i> is closer to the true line, allowing it to select the next pixel without any rounding .
Type of Arithmetic Used	Uses floating-point arithmetic (e.g., float or double). It requires floating-point addition and a rounding operation at every single step.	Uses integer arithmetic only . All calculations (after an initial setup) involve only integer addition, subtraction, and bit-shifting (which is a very fast multiplication by 2). It completely avoids division and floating-point numbers in its main loop.
Accuracy and Speed	Speed: Slower. Floating-point operations (especially addition and rounding) are significantly more computationally expensive for a CPU than simple integer operations. Accuracy: Less accurate. It is prone to cumulative rounding errors . Over very long lines, the small error from rounding at each step can add up, causing the plotted line to drift away from its true path.	Speed: Much faster and highly efficient. Integer operations are computationally "cheap" and are a native strength of the CPU. Accuracy: More accurate and stable. Because it uses only integer calculations, it is not subject to the cumulative floating-point rounding errors that affect DDA.

ADVANTAGES AND LIMITATIONS

Here is a summary of the pros and cons for each algorithm.

DIGITAL DIFFERENTIAL ANALYZER (DDA)

- **Advantages:**
 - **Conceptually Simpler:** Its logic is easier to understand, as it's a direct implementation of the line equation $y = mx + b$.³

- **Easy to Implement:** The code for DDA is often more straightforward to write for a beginner.
- **Limitations:**
 - **Slow Performance:** It is computationally expensive due to its reliance on floating-point arithmetic and rounding at every step.⁴
 - **Inaccurate:** It suffers from cumulative rounding errors, which can make long lines visibly inaccurate.⁵
 - **Not Ideal for Hardware:** The use of floating-point units makes it less efficient for implementation directly in graphics hardware compared to integer-based algorithms.⁶

BRESENHAM'S LINE ALGORITHM

- **Advantages:**
 - **Extremely Fast:** It is one of the most-efficient line-drawing algorithms because it uses only simple integer operations.⁷
 - **Highly Accurate:** It avoids rounding errors, and the plotted line stays as close as possible to the true mathematical line.⁸
 - **Hardware Friendly:** Its all-integer logic makes it ideal for direct implementation in graphics hardware (like GPUs) and plotters.⁹
- **Limitations:**
 - **More Complex Derivation:** While the final algorithm is simple to *code*, the mathematical logic used to derive the integer decision parameter is more complex to understand than DDA's simple slope concept.

In summary, while DDA is easier to understand, **Bresenham's algorithm is superior in every practical aspect** (speed, accuracy, and efficiency) and is the standard method used in most modern graphics hardware and software.

QUESTION NO 2:

A key point to understand from the start is that these two algorithms are **extremely similar**. In fact, Bresenham's circle algorithm is often considered an optimized derivative of the Midpoint algorithm. They are frequently used interchangeably, and their final implementations are nearly identical, resulting in the same efficiency and output.

The main difference lies in the *derivation* of their decision logic.

COMPARISON: MIDPOINT VS. BRESENHAM CIRCLE ALGORITHM

Criteria	Midpoint Circle Algorithm	Bresenham's Circle Algorithm
How It Selects the Next Pixel	It uses a decision parameter derived by evaluating the circle equation $f(x, y) = x^2 + y^2 - r^2$ at the midpoint between the two potential next pixels. * If the midpoint is <i>inside</i> the circle, the "East" pixel is chosen. * If the midpoint is <i>outside</i> the circle, the "South-East" pixel is chosen.	It also uses an integer decision parameter (or "error term"). This parameter is updated incrementally at each step. * The <i>sign</i> of this parameter determines which of the two potential next pixels is closer to the true circle. * This decision avoids the explicit "midpoint" concept in its final logic, focusing only on updating the error term.
Use of Symmetry	Identical. Both algorithms calculate the pixel positions for only one octant (a 45° segment) of the circle. They then use 8-way symmetry to reflect these points across the X, Y, and $y=x$ axes to draw the complete circle with minimal computation.	Identical. It also calculates points for only one octant and uses 8-way symmetry to plot the remaining seven octants. This is a massive performance optimization that both algorithms share.
Type of Arithmetic Used	Integer arithmetic only. After the initial setup, the algorithm's main loop only uses simple integer addition and subtraction (and bit-shifting, a fast multiplication by 2). It completely avoids floating-point numbers, division, and square roots.	Integer arithmetic only. Like the Midpoint algorithm, its core strength is that its main loop consists of only integer addition and subtraction. This makes it incredibly fast and efficient for hardware implementation.
Efficiency and Implementation	Efficiency: Extremely high. It is one of the most efficient ways to draw a circle, as integer operations are very "cheap" for a CPU. Implementation: The logic is straightforward once the derivation of the decision parameter is understood.	Efficiency: Identical to the Midpoint algorithm. It is also extremely fast and efficient. Implementation: Also straightforward. Because its logic is almost the same as the Midpoint algorithm, the final code for both is virtually interchangeable, often differing only in the name of the variables.

SUMMARY: WHAT'S THE "REAL" DIFFERENCE?

Think of it this way:

- **Midpoint Algorithm** is the *concept* or *derivation*. It asks the question, "Is the midpoint between my next two possible pixels inside or outside the circle?"

- **Bresenham's Algorithm** is the *optimized implementation*. It takes the logic from the Midpoint algorithm and converts it into a highly efficient decision parameter (error term) that is updated with simple integer addition at each step.

For your assignment, you can state that **Bresenham's algorithm is a highly optimized version of the Midpoint circle algorithm**. While their conceptual derivations differ slightly, they both achieve the same goal using only integer arithmetic and 8-way symmetry, making them equally fast and efficient in practice.

QUESTION NO 3

The main difference is their stopping condition:

- **Boundary Fill** checks if a pixel is a **boundary color** and *stops* filling if it is.
- **Flood Fill** checks if a pixel is an **original interior color** and *continues* filling if it is.

COMPARISON: BOUNDARY FILL VS. FLOOD FILL

Criteria	Boundary Fill Algorithm	Flood Fill Algorithm (Seed Fill)
Working Method & Seed Point	Starts from a seed point <i>inside</i> the region. It checks neighboring pixels. If a pixel is NOT the boundary_color and NOT the fill_color, it fills that pixel and recursively checks its neighbors.	Starts from a seed point <i>inside</i> the region. It checks neighboring pixels. If a pixel IS the target_color (the original interior color), it changes it to the fill_color and recursively checks its neighbors.
Connectivity (4-way & 8-way)	Both methods are used. * 4-way: Checks the four pixels: (x+1, y), (x-1, y), (x, y+1), (x, y-1). This is faster but won't fill past diagonal gaps. * 8-way: Checks all eight adjacent pixels, including diagonals. This is more thorough.	Both methods are used. * 4-way: Checks the four cardinal neighbors. * 8-way: Checks all eight neighbors, including diagonals. This ensures regions connected only by a corner are also filled.
Memory Use & Recursion	It is most easily implemented using recursion . This makes the code simple but uses the call stack for memory. Limitation: A very large region can cause a stack overflow due to deep recursion. It can be implemented non-recursively using an explicit stack (DFS) or queue (BFS) to prevent this.	It is also most easily implemented using recursion . Limitation: Like Boundary Fill, it suffers from a high risk of stack overflow on large regions. The non-recursive stack/queue implementation is safer and preferred in professional software.
Limitations	Requires a single, continuous boundary color. If there is even a one-pixel gap in the boundary, the fill will "leak" out. It	Requires a single, uniform interior color. It won't work on a region with a gradient or texture, as it would stop at

	may also fail if the fill color is the same as the interior color.	the first pixel that wasn't the exact target_color. It can be slower as it has to replace the color of every pixel <i>inside</i> the region.
Application	Best for filling shapes that are defined by a clear, single-color border . <i>Example:</i> Filling a circle or polygon that has been drawn with a black outline.	Best for replacing one solid color with another , regardless of the boundary's color. <i>Example:</i> The "Paint Bucket" tool in MS Paint. Changing a blue sky (all one color) to red, even if the sky is bordered by clouds, mountains, and trees (all different colors).

SUMMARY

- Use **Boundary Fill** when your shape is defined by a clean, consistent-colored **fence**.
- Use **Flood Fill** when you want to replace a large, connected area of a single **color** (like a patch of grass).